

¿Qué es PPM?

Prediction by Partial Matching: compresión sin pérdida basada en contexto.

PPM estima la probabilidad del próximo símbolo mirando los símbolos inmediatamente anteriores.

Ejemplo intuitivo

... A B → ¿qué viene?



Si antes vimos

AB → R

AB → R

AB → C



**R será más probable
que C después de AB**

PPM predice el próximo símbolo a partir del contexto reciente.

Prediction by Partial Matching + Codif. Aritmética

PPM-C: modelo probabilístico

Recibe un símbolo y elige una distribución según el contexto: O2, O1, O0 u O-1. Luego actualiza frecuencias.



Aritmético: coder

Toma cada evento (ESC o símbolo) y refina un único intervalo [low, high). Puede emitir bits por renormalización.



Salida

Se obtiene un stream binario. El flush cierra los bits pendientes del intervalo residual.

PPM decide probabilidades; el codificador aritmético convierte esas probabilidades en bits.

Características clave de PPM

Sin pérdida

Reconstruye exactamente el texto original.

Basado en contexto

Las probabilidades dependen de símbolos previos.

Adaptativo

Aprende mientras procesa: codifica y luego actualiza.

Múltiples órdenes

Prueba $O_2 \rightarrow O_1 \rightarrow O_0 \rightarrow O-1$.

ESC/backoff

Si no conoce el símbolo, baja de contexto.

Separado del coder

Modela probabilidades; otro coder genera bits.

Órdenes de contexto: de lo específico a lo general

Ejemplo: queremos codificar el próximo (R) símbolo después de “...AB” - AB**R**ACADABRA



Regla: comenzar en el mayor contexto disponible. Si el símbolo no está, emitir ESC y bajar de orden.

El papel de ESC: bajar de contexto

ESC no es texto original

Es un evento interno: significa “este símbolo no puede codificarse en este contexto”.

No se acumula como símbolo real

En PPM-C su peso es virtual: $f(\text{ESC})=U$, donde U es el número de símbolos distintos del contexto.

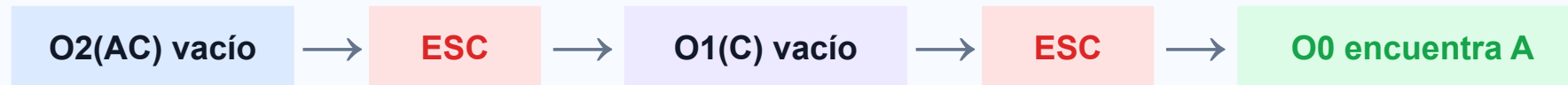
Permite sincronizar decoder

El decoder también lee ESC y sabe que debe bajar al siguiente orden.

Ejemplo de descenso

ABRAC A

↑
carácter a codificar



PPM-C: cómo calcula probabilidades

En cada contexto, solo se cuentan símbolos reales. ESC se calcula con un peso virtual.

total = suma de frecuencias
reales
U = número de símbolos
distintos

$$P(x) = \text{freq}(x) / (\text{total} + U)$$

$$P(\text{ESC}) = U / (\text{total} + U)$$

Entrada	Peso	Prob
ESC	U=3*	3/7
A	2	2/7
B	1	1/7
C	1	1/7

* El peso de ESC no se acumula: es virtual y vale U.

Primero se codifica con el estado actual; después se actualizan las tablas.

Algoritmo PPM-C: etapa de modelado

1

Leer carácter

Tomar el próximo símbolo de entrada.

2

Elegir contexto

Comenzar por el mayor orden disponible.

3

Buscar símbolo

Si está, registrar evento símbolo + probabilidad.

4

Si no está: ESC

Registrar ESC, bajar de orden y repetir.

5

Actualizar tablas

Al hallar el símbolo real: O2, O1 y O0.

PPM-C produce una secuencia de eventos y probabilidades; todavía no produce bits finales.

Codificación aritmética: convertir probabilidades en bits

El codificador aritmético toma cada evento generado por PPM-C y refina un único intervalo.

1. Evento + distribución

Ej.: A@O0 con intervalo acumulado $[3/6, 4/6)$.

Donde

$C_{\text{low}} = 3/6$

$C_{\text{high}} = 4/6$



2. Actualizar [L,H)

Al iniciar el codificador: $L=0, H=1$

$\text{range} = H - L$

$\text{newL} = L + \text{range} \cdot C_{\text{low}}$

$\text{newH} = L + \text{range} \cdot C_{\text{high}}$

$L \leftarrow \text{newL}, H \leftarrow \text{newH}$



3. Renormalizar

Aplicar E1/E2/E3 repetidamente hasta que ninguna regla aplique.

Fórmula clave

Nuevo intervalo = $[L + \text{range} \cdot C_{\text{low}} , L + \text{range} \cdot C_{\text{high}})$

La renormalización puede emitir bits antes del final, pero el flush cierra el bloque.

Reglas de renormalización: E1, E2 y E3

Regla	Condición	Acción	Idea
E1	$H \leq 0.5$	Emitir 0 $L=2L, H=2H$ Si existe pendiente, 1 x c/bit pendiente	Todo el intervalo está en la mitad inferior.
E2	$L \geq 0.5$	Emitir 1 $L=2L-1, H=2H-1$ Si existe pendiente, 0 x c/bit pendiente	Todo el intervalo está en la mitad superior.
E3	$0.25 \leq L < 0.5$ y $0.5 < H \leq 0.75$	No emitir pendientes++ $L=2L-0.5, H=2H-0.5$	Zona central: todavía no se puede decidir 0 o 1.
—	Ninguna	Esperar al siguiente evento	No hay todavía un prefijo binario seguro.

Aplicar estas reglas repetidamente hasta que ninguna sea aplicable.

Cierre del bloque: flush y condición de parada

Flush del encoder

Al finalizar, se evalúa el flush. Y se emiten los bits correspondientes para cerrar codificación aritmética.

Decoder: no deshace flush

Usa esos bits como parte del valor V y se detiene por longitud conocida o por un símbolo EOF.

En este ejemplo

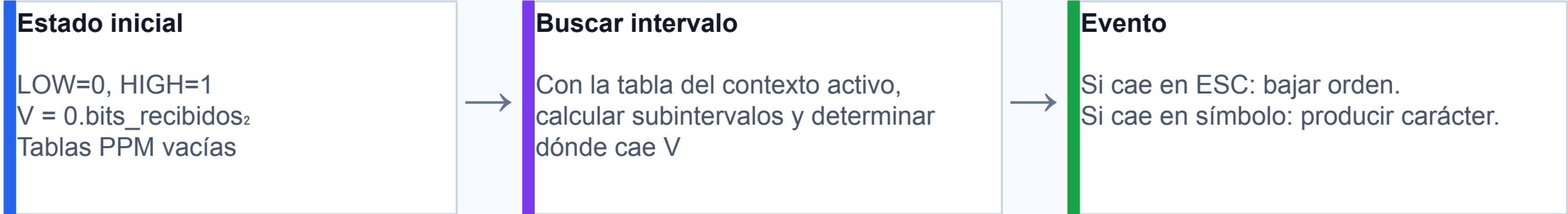
Usaremos longitud conocida: $N=11$. Así mantenemos 0-1 con 64 símbolos.

Regla	Acción
Low < 0.25	Pendientes = Pendientes + 1 Emitir 0 Por cada bit en pendiente Emitir el complemento 0 → 1
Low ≥ 0.25	Pendientes = Pendientes + 1 Emitir 1 Por cada bit en pendiente Emitir el complemento 1 → 0

Alternativa: EOF

Si se agrega un símbolo EOF explícito, el alfabeto pasa de 64 a 65 símbolos.

Decodificación: usar V para descubrir el símbolo



Renormalización en el decoder

Regla	Condición	Acción
E1	$H \leq 0.5$	consumir 0 $L=2L, H=2H$ $V=2V$
E2	$L \geq 0.5$	consumir 1 $L=2L-1, H=2H-1$ $V=2V-1$
E3	$0.25 \leq L < 0.5$ y $0.5 < H \leq 0.75$	renormaliza, sin bit fijo $L=2L-0.5, H=2H-0.5$ $V=2V-0.5$

El decoder reconstruye las tablas PPM a medida que recupera caracteres. Las tablas no se transmiten.

Codificación y decodificación son procesos espejo

Codificación

- Conoce el símbolo de entrada
- Símbolo $\rightarrow$ subintervalo
- E1/E2 emiten bits
- E3 genera pendientes
- Actualiza PPM después del símbolo

Decodificación

- Tiene que descubrir el símbolo
- $V \rightarrow$ subintervalo
- E1/E2 consumen bits
- E3 renormaliza V junto con L/H
- Actualiza PPM después de reconstruirlo

El encoder elige el intervalo. El decoder usa el valor codificado para descubrir qué intervalo había elegido el encoder.

Puente al ejemplo: ABRACADABRA

A B R A C A D A B R A

Primera etapa

PPM-C: recorrer la palabra, generar ESC/símbolos, calcular probabilidades y actualizar tablas.

Segunda etapa

Aritmético: tomar esos eventos, refinar intervalos, aplicar E1/E2/E3 y cerrar con flush.

A continuación, veremos el mismo proceso paso a paso en el Excel: tablas PPM-C → intervalos → bits → decodificación.